Bio.pairwise2 module

Pairwise sequence alignment using a dynamic programming algorithm.

This provides functions to get global and local alignments between two sequences. A global alignment finds the best concordance between all characters in two sequences. A local alignment finds just the subsequences that align the best. Local alignments must have a positive score to be reported and they will not be extended for 'zero counting' matches. This means a local alignment will always start and end with a positive counting match.

When doing alignments, you can specify the match score and gap penalties. The match score indicates the compatibility between an alignment of two characters in the sequences. Highly compatible characters should be given positive scores, and incompatible ones should be given negative scores or 0. The gap penalties should be negative.

The names of the alignment functions in this module follow the convention <alignment type>XX where <alignment type> is either "global" or "local" and XX is a 2 character code indicating the parameters it takes. The first character indicates the parameters for matches (and mismatches), and the second indicates the parameters for gap penalties.

The match parameters are:

```
CODE  DESCRIPTION
x     No parameters. Identical characters have score of 1, otherwise 0.
m     A match score is the score of identical chars, otherwise mismatch
      score.
d     A dictionary returns the score of any pair of characters.
c     A callback function returns scores.
```

The gap penalty parameters are:

```
CODE  DESCRIPTION
x     No gap penalties.
s     Same open and extend gap penalties for both sequences.
d     The sequences have different open and extend gap penalties.
c     A callback function returns the gap penalties.
```

All the different alignment functions are contained in an object `align`. For example:

```
>>> from Bio import pairwise2
>>> alignments = pairwise2.align.globalxx("ACCGT", "ACG")
```

will return a list of the alignments between the two strings. For a nice printout, use the `format_alignment` method of the module:

```
>>> from Bio.pairwise2 import format_alignment
>>> print(format_alignment(*alignments[0]))
ACCGT
| ||
A-CG-
  Score=3
```

All alignment functions have the following arguments:

- Two sequences: strings, Biopython sequence objects or lists. Lists are useful for supplying sequences which contain residues that are encoded by more than one letter.
- `penalize_extend_when_opening`: boolean (default: False). Whether to count an extension penalty when opening a gap. If false, a gap of 1 is only penalized an "open" penalty, otherwise it is penalized "open+extend".
- `penalize_end_gaps`: boolean. Whether to count the gaps at the ends of an alignment. By default, they are counted for global alignments but not for local ones. Setting `penalize_end_gaps` to (boolean, boolean) allows you to specify for the two sequences separately whether gaps at the end of the alignment should be counted.
- `gap_char`: string (default: `'-'`). Which character to use as a gap character in the alignment returned. If your input sequences are lists, you must change this to `['-']`.
- `force_generic`: boolean (default: False). Always use the generic, non-cached, dynamic programming function (slow!). For debugging.
- `score_only`: boolean (default: False). Only get the best score, don't recover any alignments. The return value of the function is the score. Faster and uses less memory.
- `one_alignment_only`: boolean (default: False). Only recover one alignment.

The other parameters of the alignment function depend on the function called. Some examples:

- Find the best global alignment between the two sequences. Identical characters are given 1 point. No points are deducted for mismatches or gaps.

  ```
  >>> for a in pairwise2.align.globalxx("ACCGT", "ACG"):
  ...     print(format_alignment(*a))
  ACCGT
  | ||
  A-CG-
    Score=3

  ACCGT
  || |
  AC-G-
    Score=3
  ```

- Same thing as before, but with a local alignment. Note that `format_alignment` will only show the aligned parts of the sequences, together with the starting positions.

```
>>> for a in pairwise2.align.localxx("ACCGT", "ACG"):
...     print(format_alignment(*a))
1 ACCG
  | ||
1 A-CG
  Score=3

1 ACCG
  || |
1 AC-G
  Score=3
```

  To restore the 'historic' behaviour of `format_alignemt`, i.e., showing also the un-aligned parts of both sequences, use the new keyword parameter `full_sequences`:

```
>>> for a in pairwise2.align.localxx("ACCGT", "ACG"):
...     print(format_alignment(*a, full_sequences=True))
ACCGT
| ||
A-CG-
  Score=3

ACCGT
|| |
AC-G-
  Score=3
```

- Do a global alignment. Identical characters are given 2 points, 1 point is deducted for each non-identical character. Don't penalize gaps.

```
>>> for a in pairwise2.align.globalmx("ACCGT", "ACG", 2, -1):
...     print(format_alignment(*a))
ACCGT
| ||
A-CG-
  Score=6

ACCGT
|| |
AC-G-
  Score=6
```

- Same as above, except now 0.5 points are deducted when opening a gap, and 0.1 points are deducted when extending it.

```
>>> for a in pairwise2.align.globalms("ACCGT", "ACG", 2, -1, -.5, -.1):
...     print(format_alignment(*a))
ACCGT
| ||
A-CG-
  Score=5

ACCGT
|| |
AC-G-
  Score=5
```

- Depending on the penalties, a gap in one sequence may be followed by a gap in the other sequence.If you don't like this behaviour, increase the gap-open penalty:

```
>>> for a in pairwise2.align.globalms("A", "T", 5, -4, -1, -.1):
...     print(format_alignment(*a))
A-

-T
  Score=-2

>>> for a in pairwise2.align.globalms("A", "T", 5, -4, -3, -.1):
...     print(format_alignment(*a))
A
.
T
  Score=-4
```

- The alignment function can also use known matrices already included in Biopython (`MatrixInfo` from `Bio.SubsMat`):

```
>>> from Bio.SubsMat import MatrixInfo as matlist
>>> matrix = matlist.blosum62
>>> for a in pairwise2.align.globaldx("KEVLA", "EVL", matrix):
...     print(format_alignment(*a))
KEVLA
 |||
-EVL-
  Score=13
```

- With the parameter `c` you can define your own match- and gap functions. E.g. to define an affine logarithmic gap function and using it:

```
>>> from math import log
>>> def gap_function(x, y):  # x is gap position in seq, y is gap length
...     if y == 0:  # No gap
...         return 0
...     elif y == 1:  # Gap open penalty
...         return -2
...     return - (2 + y/4.0 + log(y)/2.0)
...
>>> alignment = pairwise2.align.globalmc("ACCCCCGT", "ACG", 5, -4,
...                                      gap_function, gap_function)
```

  You can define different gap functions for each sequence. Self-defined match functions must take the two residues to be compared and return a score.

To see a description of the parameters for a function, please look at the docstring for the function via the help function, e.g. type `help(pairwise2.align.localds)` at the Python prompt.

---

class **Bio.pairwise2.identity_match**(*match=1*, *mismatch=0*)

Bases: `object`

Create a match function for use in an alignment.

match and mismatch are the scores to give when two residues are equal or unequal. By default, match is 1 and mismatch is 0.

> **__init__**(*self*, *match=1*, *mismatch=0*)
>
> Initialize the class.

> **__call__**(*self*, *charA*, *charB*)
>
> Call a match function instance already created.

class **Bio.pairwise2.dictionary_match**(*score_dict*, *symmetric=1*)

Bases: `object`

Create a match function for use in an alignment.

Attributes:

- score_dict - A dictionary where the keys are tuples (residue 1, residue 2) and the values are the match scores between those residues.
- symmetric - A flag that indicates whether the scores are symmetric.

> **__init__**(*self*, *score_dict*, *symmetric=1*)
>
> Initialize the class.

> **__call__**(*self*, *charA*, *charB*)
>
> Call a dictionary match instance already created.

class **Bio.pairwise2.affine_penalty**(*open*, *extend*, *penalize_extend_when_opening=0*)

Bases: `object`

Create a gap function for use in an alignment.

> **__init__**(*self*, *open*, *extend*, *penalize_extend_when_opening=0*)
>
> Initialize the class.

> **__call__**(*self*, *index*, *length*)
>
> Call a gap function instance already created.

Bio.pairwise2.calc_affine_penalty(*length*, *open*, *extend*, *penalize_extend_when_opening*)

Calculate a penality score for the gap function.

Bio.pairwise2.print_matrix(*matrix*)

Print out a matrix for debugging purposes.

Bio.pairwise2.format_alignment(*align1*, *align2*, *score*, *begin*, *end*, *full_sequences=False*)

Format the alignment prettily into a string.

IMPORTANT: Gap symbol must be "-" (or ['-'] for lists)!

Since Biopython 1.71 identical matches are shown with a pipe character, mismatches as a dot, and gaps as a space.

Prior releases just used the pipe character to indicate the aligned region (matches, mismatches and gaps).

Also, in local alignments, if the alignment does not include the whole sequences, now only the aligned part is shown, together with the start positions of the aligned subsequences. The start positions are 1-based; so start position n is the n-th base/amino acid in the *un-aligned* sequence.

NOTE: This is different to the alignment's begin/end values, which give the Python indices (0-based) of the bases/amino acids in the *aligned* sequences.

If you want to restore the 'historic' behaviour, that means displaying the whole sequences (including the non-aligned parts), use `full_sequences=True`. In this case, the non-aligned leading and trailing parts are also indicated by spaces in the match-line.